AVLSI Lab Report

Experiment 8

Group No. : 20
Name : Himesh Kundal, Ronit Gupta
Roll No. : 23EC10030, 23EC30064
Date : 5nd April, 2026

1 Problem Statement

1.1 Part A: 4x1 Multiplexer Design

- Design a 4x1 Multiplexer (MUX) using dataflow modeling in Verilog.
- Verify the functionality using a testbench by iterating through all possible select line combinations for two distinct data patterns.

1.2 Part B: 4-Bit Adder Design

- Obj-2(a): Implement a 4-bit Adder using dataflow modeling and the concatenation operator to handle the carry-out bit naturally.
- Obj-2(b): Implement a 4-bit Carry-Lookahead Adder (CLA) in both dataflow and structural modeling styles.
- Verify all implementations using testbenches covering zero inputs, simple additions, half-range carry propagation, overflow, and carry-in injection cases.

2 Procedure

2.1 4x1 Multiplexer Design

A 4x1 Multiplexer selects one of four single-bit inputs (d[3:0]) based on a 2-bit select signal (s[1:0]) and routes it to the output y. The implementation uses dataflow modeling with a single assignment statement that exploits Verilog's ability to use a signal as an array index:

```
assign y = d[s];
```

This single line completely describes the MUX function. The select signal s directly indexes into the data bus d, picking out the appropriate bit. The testbench applied two data patterns (1010 and 1100) and swept the select line from 00 to 11 for each, covering all four selection cases per pattern.

2.2 4-Bit Adder (Dataflow with Concatenation)

The 4-bit adder uses Verilog's built-in arithmetic addition operator combined with the concatenation operator to capture both the 4-bit sum and the carry-out in a single expression:

```
assign {Cout, Sum} = A + B + Cin;
```

The left-hand side concatenates the Cout and Sum signals into a 5-bit target, so the addition result (which can be at most 5 bits wide) is split automatically with the MSB going to Cout and the lower 4 bits going to Sum. This is the most concise way to describe a full adder at the dataflow level.

2.3 4-Bit Carry-Lookahead Adder (CLA)

A Carry-Lookahead Adder eliminates the sequential carry propagation delay of a ripple carry adder by computing all carry signals simultaneously in one logic stage. The key idea is to define two intermediate functions per bit:

- Generate (G[i] = A[i] AND B[i]): Bit i will generate a carry regardless of any incoming carry.
- Propagate (P[i] = A[i] XOR B[i]): Bit i will pass an incoming carry to the next bit.

Using these, all four carry bits C[1] through C[4] are expressed as functions of only the original inputs A, B, and Cin, with no dependency between carry bits. This means all carries are resolved in a fixed, small gate delay irrespective of word length.

Two implementations were developed:

- Dataflow CLA: All Generate, Propagate, carry, and sum equations are written as flat assign statements in a single module.

- Structural CLA: The design is decomposed into three reusable submodules. pg_unit computes P and G per bit, cla_block implements the lookahead carry equations, and sum_unit computes each output sum bit. A top-level module (adder_4bit_cla) instantiates these units and wires them together.

3 Codes and Results

3.1 Part A: 4x1 Multiplexer (Dataflow)

Source files located in exp3/mux4to1/

Multiplexer (mux4to1.v)

```
module mux4to1 (
       input  [3:0] d,
       input  [1:0] s,
       output       y
);
```

```
	// Dataflow modeling using bit-selection

	assign y = d[s];

endmodule
```

Testbench (testbench.v)

```
`timescale 1ns/1ps



module testbench;

	reg  [3:0] d;

	reg  [1:0] s;

	wire       y;



	mux4to1 uut (.d(d), .s(s), .y(y));



	integer i;

	initial begin

	$dumpfile("dump.vcd");

	$dumpvars(0, testbench);
```

```
        $display("Time\tS\tD\t\tY");

        $monitor("%t\t%b\t%b\t%b", $time, s, d, y);



        d = 4'b1010;

        for (i = 0; i < 4; i = i + 1) begin s = i; #10; end



        d = 4'b1100;

        for (i = 0; i < 4; i = i + 1) begin s = i; #10; end



        $finish;

        end

endmodule
```

Makefile

```
OUT = mux4to1

SRC = mux4to1.v testbench.v

.PHONY: all compile run wave clean

all: run
```

```
compile:
	iverilog -o $(OUT) $(SRC)
run: compile
	vvp $(OUT)
wave: compile
	vvp $(OUT)
	gtkwave dump.vcd
clean:
	rm -f $(OUT) *.vcd
```

Simulation Result

Two data patterns were applied while the select line swept from 00 to 11:

Time (ns)	S[1:0]	D[3:0]	Y
0	00	1010	0

10	01	1010	1
20	10	1010	0
30	11	1010	1
40	00	1100	0
50	01	1100	0
60	10	1100	1
70	11	1100	1

3.2 Part B(a): 4-Bit Adder (Dataflow and Concatenation)

Source files located in exp3/adder_4bit/

Adder (adder_4bit.v)

```
module adder_4bit (
	input  [3:0] A, B,
	input        Cin,
	output [3:0] Sum,
	output       Cout
);
	// Dataflow modeling using concatenation operator
	assign {Cout, Sum} = A + B + Cin;
endmodule
```

Testbench (testbench.v)

```
`timescale 1ns/1ps


module testbench;
	reg  [3:0] A, B;
	reg        Cin;
```

```
wire [3:0] Sum;

wire       Cout;



adder_4bit uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));



initial begin

$dumpfile("dump.vcd");

$dumpvars(0, testbench);

 $display("Time\tA\tB\tCin\tSum\tCout");

 $monitor("%t\t%b\t%b\t%b\t%b\t%b",

       $time, A, B, Cin, Sum, Cout);



A = 4'b0000; B = 4'b0000; Cin = 0; #10;

A = 4'b0001; B = 4'b0001; Cin = 0; #10;

A = 4'b0111; B = 4'b0001; Cin = 0; #10;

A = 4'b1111; B = 4'b1111; Cin = 0; #10;

A = 4'b1010; B = 4'b0101; Cin = 1; #10;

$finish;
```

```
        end

endmodule
```

Makefile

```
OUT = adder_4bit

SRC = adder_4bit.v testbench.v

.PHONY: all compile run wave clean

all: run

compile:

        iverilog -o $(OUT) $(SRC)

run: compile

        vvp $(OUT)

wave: compile

        vvp $(OUT)

        gtkwave dump.vcd

clean:

        rm -f $(OUT) *.vcd
```

Simulation Result

Time (ns)	A	B	Cin	Sum	Cout
0	0000	0000	0	0000	0
10	0001	0001	0	0010	0
20	0111	0001	0	1000	0
30	1111	1111	0	1110	1
40	1010	0101	1	0000	1

3.3 Part B(b): 4-Bit CLA (Dataflow)

Source files located in exp3/adder_4bit_cla_dataflow/

CLA Adder (adder_4bit_cla.v)

```
module adder_4bit_cla (
	input  wire [3:0] A, B,
	input  wire Cin,
	output wire [3:0] Sum,
	output wire Cout
);
	wire [3:0] P, G;
	wire [4:1] C;


	// Generate and Propagate terms
	assign P = A ^ B;
	assign G = A & B;


	// Carry Lookahead equations
	assign C[1] = G[0] | (P[0] & Cin);
```

```
    assign C[2] = G[1] | (P[1] & G[0]) | (P[1] & P[0] & Cin);

    assign C[3] = G[2] | (P[2] & G[1]) | (P[2] & P[1] & G[0])

                  | (P[2] & P[1] & P[0] & Cin);

    assign C[4] = G[3] | (P[3] & G[2]) | (P[3] & P[2] & G[1])

                  | (P[3] & P[2] & P[1] & G[0])

                  | (P[3] & P[2] & P[1] & P[0] & Cin);



    // Sum computation

    assign Sum[0] = P[0] ^ Cin;

    assign Sum[1] = P[1] ^ C[1];

    assign Sum[2] = P[2] ^ C[2];

    assign Sum[3] = P[3] ^ C[3];



    assign Cout = C[4];

endmodule
```

Testbench (testbench.v)

```
`timescale 1ns/1ps
```

```
module testbench;
       reg  [3:0] A, B;
       reg        Cin;
       wire [3:0] Sum;
       wire       Cout;

       adder_4bit_cla uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));

       initial begin
       $dumpfile("dump.vcd");
       $dumpvars(0, testbench);
        $display("Time\tA\tB\tCin\tSum\tCout");
        $monitor("%t\t%b\t%b\t%b\t%b\t%b",
              $time, A, B, Cin, Sum, Cout);

       A = 4'b0000; B = 4'b0000; Cin = 0; #10;
       A = 4'b0001; B = 4'b0001; Cin = 0; #10;
```

```
        A = 4'b0111; B = 4'b0001; Cin = 0; #10;

        A = 4'b1111; B = 4'b1111; Cin = 0; #10;

        A = 4'b1010; B = 4'b0101; Cin = 1; #10;

        $finish;

        end

endmodule
```

Makefile

```
OUT = adder_4bit_cla

SRC = adder_4bit_cla.v testbench.v

.PHONY: all compile run wave clean

all: run

compile:

        iverilog -o $(OUT) $(SRC)

run: compile

        vvp $(OUT)

wave: compile

        vvp $(OUT)

        gtkwave dump.vcd
```

```
clean:
	rm -f $(OUT) *.vcd
```

Simulation Result

The dataflow CLA produces results identical to the concatenation adder for all test vectors, confirming functional correctness:

Time (ns)	A	B	Cin	Sum	Cout
0	0000	0000	0	0000	0
10	0001	0001	0	0010	0
20	0111	0001	0	1000	0
30	1111	1111	0	1110	1
40	1010	0101	1	0000	1

3.4 Part B(b): 4-Bit CLA (Structural)

Source files located in exp3/adder_4bit_cla_structural/

CLA Adder (adder_4bit_cla.v)

```
// Propagate and Generate unit

module pg_unit (

       input  wire a, b,

       output wire p, g

);

       assign p = a ^ b;

       assign g = a & b;

endmodule



// Carry Lookahead Logic (CLL) block

module cla_block (
```

```
	input  wire [3:0] p, g,
	input  wire cin,
	output wire [4:1] c
);
	assign c[1] = g[0] | (p[0] & cin);
	assign c[2] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & cin);
	assign c[3] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0])
                     | (p[2] & p[1] & p[0] & cin);
	assign c[4] = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1])
                     | (p[3] & p[2] & p[1] & g[0])
                     | (p[3] & p[2] & p[1] & p[0] & cin);
endmodule


// Sum unit
module sum_unit (
	input  wire p, c,
	output wire sum
);
	assign sum = p ^ c;
```

```
endmodule



// Top level structural 4-bit CLA

module adder_4bit_cla (

       input  wire [3:0] A, B,

       input  wire Cin,

       output wire [3:0] Sum,

       output wire Cout

);

       wire [3:0] P, G;

       wire [4:0] C;



       assign C[0] = Cin;



       pg_unit pg0 (A[0], B[0], P[0], G[0]);

       pg_unit pg1 (A[1], B[1], P[1], G[1]);

       pg_unit pg2 (A[2], B[2], P[2], G[2]);

       pg_unit pg3 (A[3], B[3], P[3], G[3]);
```

```
	cla_block cll (P, G, C[0], C[4:1]);


	sum_unit s0 (P[0], C[0], Sum[0]);
	sum_unit s1 (P[1], C[1], Sum[1]);
	sum_unit s2 (P[2], C[2], Sum[2]);
	sum_unit s3 (P[3], C[3], Sum[3]);


	assign Cout = C[4];
endmodule
```

Testbench (testbench.v)

```
`timescale 1ns/1ps


module testbench;
	reg  [3:0] A, B;
	reg        Cin;
	wire [3:0] Sum;
```

```
wire       Cout;



adder_4bit_cla uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));



initial begin

$dumpfile("dump.vcd");

$dumpvars(0, testbench);

 $display("Time\tA\tB\tCin\tSum\tCout");

 $monitor("%t\t%b\t%b\t%b\t%b\t%b",

       $time, A, B, Cin, Sum, Cout);



A = 4'b0000; B = 4'b0000; Cin = 0; #10;

A = 4'b0001; B = 4'b0001; Cin = 0; #10;

A = 4'b0111; B = 4'b0001; Cin = 0; #10;

A = 4'b1111; B = 4'b1111; Cin = 0; #10;

A = 4'b1010; B = 4'b0101; Cin = 1; #10;

$finish;

end
```

```
endmodule
```

Makefile

```
OUT = adder_4bit_cla

SRC = adder_4bit_cla.v testbench.v

.PHONY: all compile run wave clean

all: run

compile:

	iverilog -o $(OUT) $(SRC)

run: compile

	vvp $(OUT)

wave: compile

	vvp $(OUT)

	gtkwave dump.vcd

clean:

	rm -f $(OUT) *.vcd
```

Simulation Result

The structural CLA results are functionally identical to both the dataflow CLA and the concatenation adder, verifying the correctness of the hierarchical module wiring:

Time (ns)	A	B	Cin	Sum	Cout
0	0000	0000	0	0000	0
10	0001	0001	0	0010	0
20	0111	0001	0	1000	0
30	1111	1111	0	1110	1
40	1010	0101	1	0000	1

4 Simulation Waveforms

4.1 4x1 Multiplexer Waveform

The waveform below shows the GTKWave output for the 4x1 MUX. Two data patterns (0xA = 1010 and 0xC = 1100) are applied while the select bus s[1:0] sweeps from 00 to 11 for each. The output y follows d[s] exactly at every transition, with d[1] and d[3] high for pattern 0xA (alternating 1-0-1-0) and d[2] and d[3] high for pattern 0xC (0-0-1-1).

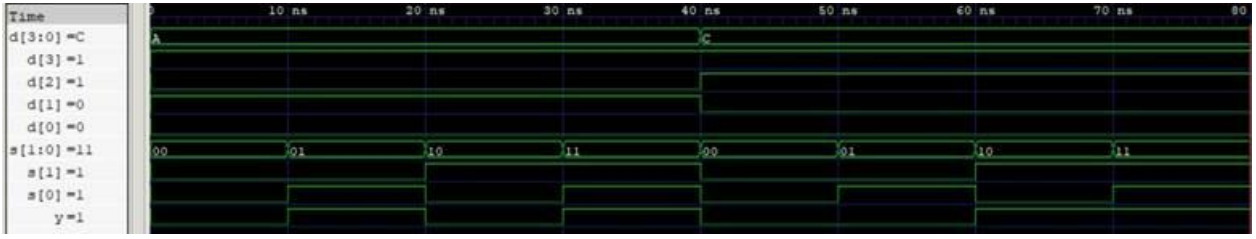

Figure 4.1 — 4x1 MUX Dataflow Simulation Waveform (GTKWave)

4.2 4-Bit Adder (Dataflow) Waveform

The waveform shows all five test vectors applied to the concatenation adder. The intermediate carry vector C[4:0] is not visible at this level of abstraction, but the Cout bit transitions correctly for the overflow (A=F, B=F giving Cout=1) and carry-in injection (A=A, B=5, Cin=1 giving Cout=1) cases. Sum[3:0] transitions match the expected arithmetic results at each 10 ns step.

Figure 4.2 — 4-Bit Dataflow Adder Simulation Waveform (GTKWave)

4.3 4-Bit CLA Adder (Dataflow) Waveform

This waveform exposes the internal intermediate signals of the dataflow CLA, including the full carry vector C[4:0], the Generate bus G[3:0], and the Propagate bus P[3:0]. These signals confirm that all carries are computed in a single combinational pass from the primary inputs. For the test case A=7, B=1 (20 ns), all carry bits C[1] through C[3] assert simultaneously, with none waiting on the previous, which is the defining property of a lookahead adder.

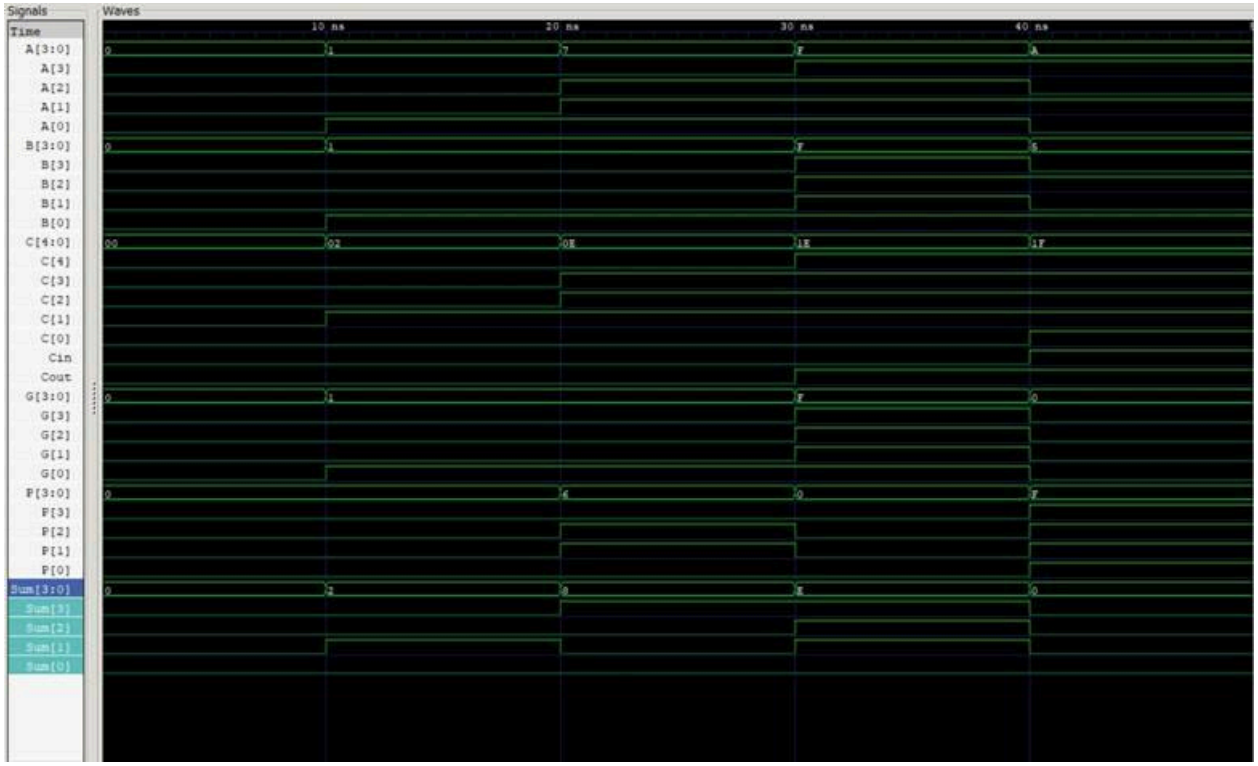

Figure 4.3 — 4-Bit CLA Dataflow Simulation Waveform (GTKWave)

4.4 4-Bit CLA Adder (Structural) Waveform

The structural CLA waveform additionally exposes the individual submodule port signals for pg0 through pg3 (each showing a, b, g, p), as well as s0 through s3 (each showing c, p, sum). This level of visibility allows direct inspection of every intermediate node in the design hierarchy. The per-bit propagate and generate values at each time step match the expected manual calculations, and the final Sum and Cout values are identical to those produced by the dataflow implementation, confirming structural correctness.

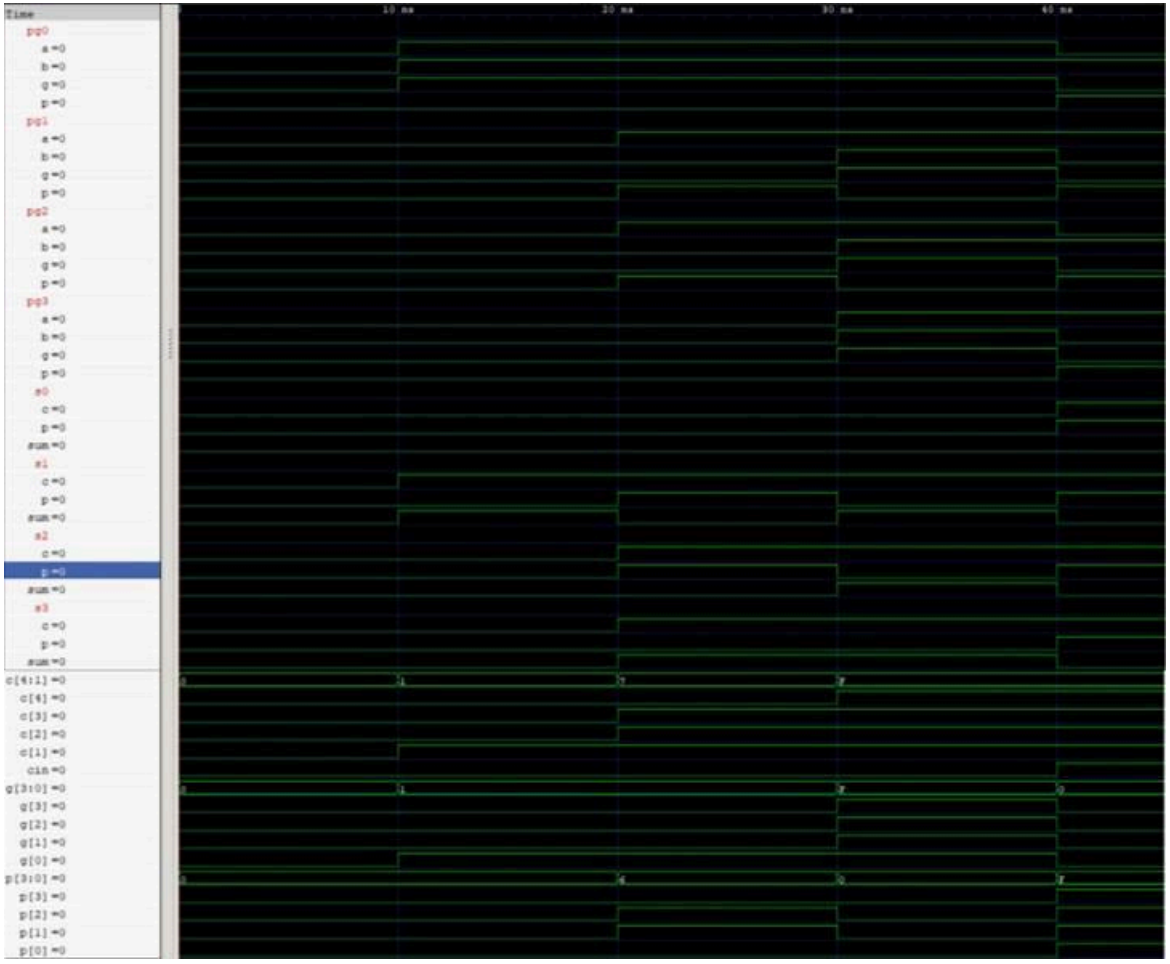

Figure 4.4 — 4-Bit CLA Structural Simulation Waveform (GTKWave)

5 Observations

5.1 4x1 Multiplexer

- For data pattern 1010, the output alternates between 0 and 1 as s increments from 00 to 11, since alternate bits of the pattern are set. This directly mirrors the bit-selection behavior of the assign y = d[s] statement.
- For data pattern 1100, the output is 0 for s = 00 and 01, then 1 for s = 10 and 11, matching the upper two bits of the pattern being set. The waveform confirms these transitions at exactly 40 ns and 60 ns.
- The dataflow single-line implementation is functionally complete. There is no need for any logic gates or case statements, because Verilog's array indexing by a variable already encodes the full 4:1 selection truth table.

5.2 4-Bit Adder (Concatenation)

- The test case A=0111, B=0001 exercises carry propagation from bit 0 through all bits to produce Sum=1000, which is the worst-case ripple scenario at the behavioral level. The operator handles it correctly in a single expression.
- The overflow case A=1111, B=1111 produces Sum=1110 with Cout=1. Mathematically 15+15=30, which is 11110 in binary. The 4-bit sum correctly captures the lower nibble (1110) and the MSB becomes Cout=1.
- The carry-in injection case A=1010, B=0101, Cin=1 gives Sum=0000 with Cout=1. Decimal 10+5+1=16, which is 10000. The lower nibble wraps to 0 and the carry-out is set, confirming Cin is correctly fed into the addition.

5.3 CLA vs. Concatenation Adder

- All three implementations (concatenation adder, dataflow CLA, structural CLA) produced identical outputs for every test vector, confirming full functional equivalence across three different levels of abstraction.
- The CLA waveform shows the G and P buses updating in the same simulation step as A and B. Carry bits C[1] through C[4] also update in the same step, with no sequential dependency visible in simulation, which is consistent with a purely combinational lookahead network.
- The structural CLA waveform exposes individual pg_unit outputs, making it easy to verify that each bit's generate and propagate values are correct before they enter the cla_block. This per-module traceability is an advantage of hierarchical structural design over a flat dataflow description.

5.4 Dataflow vs Structural CLA

- The dataflow CLA and the structural CLA describe the same logic but differ in how the hierarchy is expressed. The dataflow version is more compact. The structural version distributes the logic across submodules, which improves readability and makes the design easier to scale to larger bit widths.
- In both cases, the carry logic has a fixed depth of four gate levels (AND-OR) regardless of which carry bit is being computed, as opposed to up to eight gate levels for a 4-bit ripple carry adder. This fixed, short delay is the primary advantage of the CLA topology.

6 Truth Tables and Verification

6.1 4x1 MUX Select Mapping

With d = 4'b1010:

S[1:0]	Selected Bit	d[s]	Expected Y	Simulated Y	Pass?
00	d[0]	0	0	0	YES
01	d[1]	1	1	1	YES
10	d[2]	0	0	0	YES
11	d[3]	1	1	1	YES

With d = 4'b1100:

S[1:0]	Selected Bit	d[s]	Expected Y	Simulated Y	Pass?
00	d[0]	0	0	0	YES
01	d[1]	0	0	0	YES
10	d[2]	1	1	1	YES
11	d[3]	1	1	1	YES

6.2 4-Bit Adder Verification Table

All three adder implementations were verified against the expected arithmetic result:

A (hex)	B (hex)	Cin	Expected Sum	Simulated Sum	Cout	Decimal Check	Status
0x0	0x0	0	0000	0000	0	0+0=0	PASS

0x1	0x1	0	0010	0010	0	1+1=2	PASS
0x7	0x1	0	1000	1000	0	7+1=8	PASS
0xF	0xF	0	1110	1110	1	15+15=30	PASS
0xA	0x5	1	0000	0000	1	10+5+1=16	PASS

6.3 CLA Carry Lookahead Manual Trace (A=0111, B=0001, Cin=0)

Manual derivation of all intermediate signals to verify the CLA logic:

Bit i	A[i]	B[i]	P[i]=A XOR B	G[i]=A AND B	C[i+1]	Sum[i]=P[i] XOR C[i]
0	1	1	0	1	C[1]=1	0 XOR 0 = 0
1	1	0	1	0	C[2]=1	1 XOR 1 = 0

2	1	0	1	0	C[3]=1	1 XOR 1 = 0
3	0	0	0	0	C[4]=0	0 XOR 1 = 1

Result: Sum = 1000, Cout = 0. Matches simulation output (7+1=8). All bits PASS.

7 Discussion

7.1 Himesh Kundal (23EC10030)

- The 4x1 MUX implementation in this experiment was remarkable in how much it condensed. The entire module is one line of dataflow Verilog, yet it is completely synthesizable and fully correct. Using the select signal as an index into the data bus is one of those Verilog idioms that feels like a shortcut but actually has a clear hardware interpretation: the synthesizer converts it into a 4-to-1 selector network just as a case statement would. The waveform confirmed that each transition in the select line produces the correct output immediately with no glitching, which is expected for a purely combinational assignment.

- Working through the concatenation adder made the handling of carry-out in Verilog feel very natural. The expression {Cout, Sum} = A + B + Cin treats the five-bit result as a unified quantity and splits it at compile time. What is interesting is that this works cleanly even for the overflow case: the full arithmetic truth is preserved in the 5-bit result and none of it is silently discarded. This is a contrast to many software languages where integer overflow wraps silently without a carry flag.
- The move from the ripple carry adder in Experiment 2 to the CLA in this experiment highlighted a real architectural progression. The ripple adder is intuitive but has a latency proportional to word length. The CLA pays a cost in logic complexity by computing longer and longer Boolean expressions for higher carry bits, but it recovers a flat, constant latency. The dataflow CLA waveform made this very visible: all carry bits C[1] through C[4] update in the same simulation timestep as the inputs, with no carry-by-carry progression visible at all.
- Writing the CLA carry equations by hand and then seeing them produce the correct results in simulation was a good exercise in Boolean expansion. The equation for C[4] is the longest and has five product terms. Tracing through the manual calculation for the test case A=0111, B=0001 confirmed that each term has a specific physical meaning: the first term fires if bit 3 itself generates a carry, the second if bit 3 propagates a carry generated at bit 2, and so on down to the last term which covers the case where the carry ripples all the way from Cin through all four propagate stages.
- The testbench design for the adder followed the same philosophy as in previous experiments: rather than testing every possible 4-bit combination, a small set of vectors was chosen to stress specific conditions. The half-range case (7+1=8) targets carry propagation through three consecutive bits. The overflow case (15+15) maximizes both sum and carry-out. The carry-in injection case (10+5+1=16) confirms that Cin reaches bit 0 correctly and its effect propagates upward. Together these five vectors provide high diagnostic coverage with minimal simulation time.

7.2 Ronit Gupta (23EC30064)

- The structural CLA waveform was the most informative output of this entire experiment. Having pg0 through pg3 and s0 through s3 individually visible in GTKWave meant that every intermediate computation in the design hierarchy could be checked directly against the expected values from a manual calculation. For the test case A=F, B=F, every pg_unit shows a=1, b=1, g=1, p=0 simultaneously, which is correct since all bits generate and none propagate. Seeing this laid out across sixteen waveform rows made the generate-heavy character of the overflow case very concrete.
- Comparing the dataflow CLA and the structural CLA side by side showed that both describe exactly the same network of gates. The dataflow version writes the carry equations directly in the top-level module. The structural version distributes them across submodules. From a synthesis perspective these two would produce very similar netlists. The structural version has a small overhead in port declarations and instantiation syntax, but it gains in clarity and reusability. If the design were to scale to 8 or 16 bits, the structural version would simply require adding more pg_unit and sum_unit instances and expanding the cla_block, which is a predictable and safe operation.
- One thing that became clear from the carry manual trace in the verification section is that the CLA carries are not independent in the sense that they share subexpressions. For example, the term p[1] AND g[0] appears in both C[2] and C[3]. In a gate-level synthesis, a good tool would share this logic and not duplicate the AND gate. This concept of logic sharing is what keeps the area overhead of CLA manageable despite the longer Boolean expressions for higher carry bits.
- The 4-to-1 MUX in Part A and the adder in Part B represent two of the most fundamental datapath building blocks in digital design. Multiplexers are used for bus arbitration, operand selection in ALUs, and routing data between functional units. Adders are the core of every arithmetic unit. Having implemented both from scratch in Verilog, including at the gate level for the adder, gives a clearer picture of how the components in a real processor datapath connect and interact.
- Seeing all three adder implementations produce identical simulation outputs was instructive. It confirmed that the level of abstraction used to describe a circuit does not affect its functional behavior, only the amount of detail the designer must specify. The synthesis tool bridges the remaining gap between a behavioral description and a physical circuit. This experiment

demonstrated that gap clearly: the concatenation adder says what the circuit should compute, the dataflow CLA says how the carry logic works, and the structural CLA says which physical blocks exist and how they are connected. All three are valid and equivalent entry points to the same underlying hardware.